以下为AI生成的图文笔记的内容

一、运行时加速方案 **00:21**

1. 多头注意力机制 01:05

- **核心作用**：在Transformer架构中用于计算QKV矩阵，通过softmax生成注意力参数输出
- **优势特点**：
  - 使模型更"聪明"：从多角度理解词间关系
  - 增强语义表达：使输出内容更人性化
- **性能代价**：
  - 显著增加KV缓存占用
  - 推理性能下降与注意力头数成正比

2. GQA分组查询注意力机制 01:52

- **优化目标**：降低KV缓存占用
- **技术变体**：
  - MQA（多查询注意力）：多个查询共享同一组键值对
  - GQA（分组查询注意力）：查询分组共享键值对
- **应用权衡**：
  - 可能损失部分精度/准确率
  - 需根据具体场景选择机制
  - 推理性能提升价值大于精度损失

3. 机制性能对比 02:21

- **对比维度**：
  - 内存占用效率
  - 计算吞吐量
  - 延迟表现
- **典型规律**：
  - 多头注意力机制精度最高但资源消耗最大
  - MQA资源效率最优但精度损失最明显
  - GQA在两者间取得平衡

4. FlashAttention技术 02:35

- **问题背景**：
  - GPU显存与计算核心间数据搬运开销大
  - 大模型参数导致频繁的显存读写
- **解决方案**：
  - 将大向量分块处理（tiling技术）
  - 小块数据驻留计算核心缓存
  - 局部结果保留在计算核心
  - 最终合并分块计算结果
- **技术优势**：
  - 减少90%以上的显存访问
  - 充分利用GPU高速缓存

5. PagedAttention技术 04:06

- **产生需求**：
  - FlashAttention导致内存碎片化
  - 变长token序列处理效率低
- **核心思想**：
  - 建立逻辑块与物理存储块的映射表

- 类似操作系统分页管理机制
- **技术效果**:
  - 消除内存碎片
  - 支持动态token长度处理
  - 与FlashAttention形成互补

6. 持续批处理 04:33

- **传统局限**:
  - 单指令多线程(SIMT)执行模式
  - 批处理需等待最慢线程完成
- **创新机制**:
  - 动态线程调度
  - 完成线程立即释放资源
  - 新请求实时填充空闲计算单元
- **性能提升**:
  - 计算资源利用率提升30-50%
  - 显著降低长尾延迟

二、知识小结

主题	核心内容	技术亮点	应用挑战
模型压缩	量化技术降低模型尺寸	**推理阶段优化**	精度与效率平衡
KV缓存机制	Transformer架构关键组件	多头注意力参数缓存	**显存占用过高**
注意力机制改进	MQA/GQA替代方案	多查询/分组注意力	场景适配性要求
Flash Attention	GPU运算优化技术	**向量分块处理**	碎片化信息管理
Page Attention	内存寻址优化	逻辑-物理块映射	变长token处理
持续批处理	GPU线程调度	动态线程填充	请求长度差异
推理框架选型	系统级解决方案	端到端优化	技术栈兼容性